

Utilização do GitHub no desenvolvimento do MentorIA

Grupo 10 - Dev InHouse

Keliven Demarchi, Rafael Lindemann e Djeniffer Machado



Unidade curricular: Desenvolvimento de Sistemas

Situação de aprendizagem: Desenvolvimento colaborativo do MentorIA

Tema: Utilização do GitHub na gestão e no desenvolvimento de uma plataforma educacional com inteligência artificial



Contextualização e problemática

O MentorIA é uma plataforma de mentoria acadêmica que utiliza inteligência artificial para oferecer trilhas e planos de estudo personalizados.

Na plataforma, o estudante poderá:

- Informar seus objetivos profissionais;
- Escolher uma trilha de aprendizagem;
- Realizar uma avaliação inicial;
- Receber um plano de estudos personalizado;
- Acompanhar atividades e seu progresso;
- Consultar conteúdos, exemplos e exercícios;
- Conversar com um mentor virtual;
- Realizar avaliações de progresso;
- Receber pontos de experiência e badges.

Antes do início da situação de aprendizagem, os requisitos, as histórias de usuário, os critérios de aceitação e o protótipo das interfaces serão disponibilizados pelo docente.

Portanto, os estudantes não precisarão levantar requisitos nem criar o protótipo. O desafio será interpretar esses artefatos e transformá-los em uma aplicação funcional.

Capacidades técnicas desenvolvidas

- Interpretar requisitos e critérios de aceitação;
- Interpretar protótipos de interfaces;
- Modelar e implementar bancos de dados relacionais;
- Definir tabelas, campos, chaves e relacionamentos;
- Desenvolver interfaces com React;
- Desenvolver APIs e regras de negócio com Node.js;
- Integrar frontend, backend e banco de dados;
- Implementar operações de cadastro, consulta, atualização e exclusão;
- Aplicar validações no frontend e no backend;
- Integrar a aplicação com serviços de inteligência artificial;
- Utilizar Git e GitHub no controle de versão;
- Organizar e acompanhar tarefas;
- Realizar revisão de código;
- Testar e validar funcionalidades;
- Documentar a implementação;
- Trabalhar colaborativamente.

Conteúdos técnicos envolvidos

- Interpretação de requisitos;
- Modelagem lógica e física do banco de dados;
- PostgreSQL;
- Sequelize;
- React;
- Node.js;
- APIs REST;
- Rotas, controladores e serviços;
- Autenticação e autorização;
- Validação de dados;
- Integração com o Gemini;
- Busca semântica com pgvector;
- RAG;
- Tratamento de erros;
- Git e GitHub;
- Branches, commits e Pull Requests;
- Testes e documentação.

Tecnologias do MentorIA

- Frontend: React;
- Backend: Node.js;
- Banco de dados: PostgreSQL;
- Mapeamento de dados: Sequelize;
- Busca vetorial: pgvector;
- Inteligência artificial: Google Gemini;
- Controle de versão e gestão: GitHub.

O PostgreSQL armazenará os dados estruturados, como usuários, trilhas, módulos, atividades, avaliações, progresso, XP, badges e histórico do chat.

O pgvector será utilizado para armazenar embeddings e recuperar conteúdos semanticamente relacionados à trilha e à pergunta do estudante.

6. Organização da situação de aprendizagem

Os estudantes serão organizados em pequenas equipes. Cada equipe ficará responsável pela implementação de um módulo do MentorIA.

Os requisitos e o protótipo serão os mesmos para todas as equipes, funcionando como referência para o desenvolvimento.

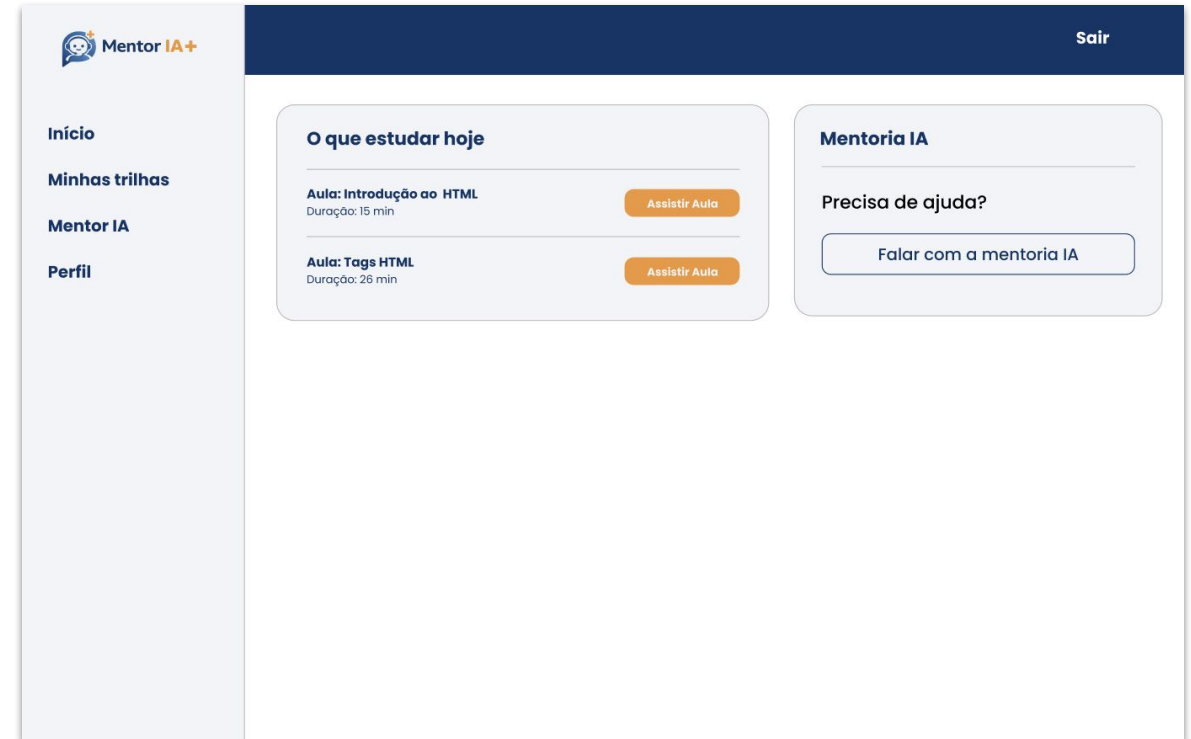
Os módulos poderão ser divididos em:

- Cadastro, autenticação e perfil;
- Trilhas e objetivos profissionais;
- Avaliação inicial;
- Plano de estudos;
- Controle de atividades e progresso;
- Chatbot com RAG;
- Avaliação de progresso;
- Gamificação com XP e badges;
- Painel inicial do estudante.

1 - Apresentação dos artefatos

- A problemática atendida pelo MentorIA;
- Os requisitos selecionados;
- As histórias de usuário;
- Os critérios de aceitação;
- O protótipo das telas;
- As tecnologias definidas;
- A arquitetura inicial do sistema;
- O repositório do projeto.

Os estudantes deverão analisar esses materiais para compreender o comportamento esperado da aplicação.



2 - Planejamento no GitHub

Os requisitos serão disponibilizados como Issues ou transformados em tarefas técnicas.

Por exemplo, o requisito “realizar avaliação inicial” poderá originar tarefas como:

- Criar as tabelas de avaliações e questões;
- Criar o modelo da avaliação no Sequelize;
- Implementar a rota para gerar a avaliação;
- Implementar o registro das respostas;
- Calcular o resultado;
- Criar a tela conforme o protótipo;
- Integrar a tela com a API;
- Validar o fluxo completo.

As Issues serão organizadas no GitHub Projects:

- Backlog;
- Pronto para iniciar;
- Em desenvolvimento;
- Em revisão;
- Em testes;
- Concluído.

Cada estudante assumirá uma ou mais tarefas, mantendo o quadro atualizado

3 - Modelagem do banco de dados

A partir dos requisitos fornecidos, os estudantes deverão criar a modelagem do banco de dados.

Poderão ser contempladas entidades como:

- Usuário;
- Perfil;
- Objetivo profissional;
- Trilha;
- Módulo;
- Atividade;
- Avaliação;
- Questão;
- Resposta;
- Progresso;
- Badge;
- Conteúdo da trilha;
- Histórico do chat.

Os estudantes deverão definir:

- Campos;
- Tipos de dados;
- Chaves primárias;
- Chaves estrangeiras;
- Relacionamentos;
- Restrições;
- Validações;
- Migrações;
- Dados iniciais necessários.

Também será implementada a estrutura necessária para utilização do pgvector.

4 - Preparação do projeto

As equipes prepararão os ambientes do frontend e do backend, configurarão o banco de dados e documentarão as instruções de execução.

O projeto deverá possuir:

- Organização de pastas;
- Configuração das variáveis de ambiente;
- Conexão com o PostgreSQL;
- Modelos do Sequelize;
- Rotas iniciais da API;
- Estrutura de componentes React;
- Tratamento básico de erros.



Faça seu login para entrar na plataforma

E-mail:

Digite seu e-mail

Senha

Digite sua senha

[Esqueci a senha](#)

Entrar

[Criar conta](#)

5 - Implementação

Cada funcionalidade será desenvolvida em uma branch específica.

Exemplos:

- feature/avaliacao-inicial;
- feature/plano-estudos;
- feature/chat-rag;
- feature/gamificacao;
- fix/contexto-chat;
- fix/geracao-avaliacao.

O estudante deverá implementar a interface de acordo com o protótipo e garantir que os dados sejam persistidos corretamente.

Os commits deverão registrar pequenas evoluções do projeto, com mensagens claras, como:

- feat: cria tabelas de trilhas e módulos;
- feat: implementa rota de avaliação inicial;
- feat: integra tela de progresso com a API;
- fix: corrige contexto recuperado pelo chatbot;
- test: valida cálculo do resultado da avaliação.

6 - Pull Requests e revisão

Ao concluir uma funcionalidade, o estudante abrirá um Pull Request vinculado à Issue.

O Pull Request deverá informar:

- O que foi implementado;
- Qual Issue foi atendida;
- Quais critérios de aceitação foram cumpridos;
- Quais testes foram realizados;
- Quais tabelas ou campos foram alterados;
- Evidências visuais da interface;
- Possíveis limitações.

Outro integrante deverá revisar o código antes da integração com a branch principal.

7 - Testes e validação

A implementação será comparada com os requisitos, critérios de aceitação e protótipo disponibilizados.

Serão verificados:

- Correspondência da interface com o protótipo;
- Funcionamento das regras de negócio;
- Integração entre frontend e backend;
- Persistência correta no banco;
- Validação dos dados;
- Tratamento de erros;
- Controle de acesso;
- Funcionamento do fluxo completo;
- Comportamento quando o serviço de IA estiver indisponível.

Os erros encontrados serão registrados como Issues do tipo bug.

8 - Integração dos módulos

Depois da implementação individual dos módulos, as equipes realizarão a integração das funcionalidades.

Será necessário verificar se:

- Os modelos utilizam relacionamentos compatíveis;
- As rotas seguem o mesmo padrão;
- A autenticação funciona em todos os módulos;
- O frontend consome corretamente a API;
- Os dados do progresso alimentam o plano, a avaliação e o chatbot;
- O sistema integrado atende aos requisitos fornecidos.

9 - Apresentação

As equipes demonstrarão:

- O módulo implementado;
- As tabelas e os relacionamentos;
- A correspondência entre requisito, protótipo e código;
- A integração entre frontend, backend e banco;
- A organização do trabalho no GitHub;
- Os testes realizados;
- Os problemas encontrados;
- As soluções adotadas.

Utilização do GitHub

README

Apresentará o projeto, as tecnologias, a organização das pastas, as variáveis de ambiente e as instruções para executar o sistema.

Issues

Registrarão as tarefas de implementação, correções, testes e melhorias relacionadas aos requisitos já definidos.

GitHub Projects

Permitirá visualizar a distribuição das tarefas e o andamento de cada módulo.

Branches

Cada funcionalidade ou correção será desenvolvida isoladamente.

Commits

Registrarão a evolução da implementação e permitirão identificar as contribuições dos estudantes.

Pull Requests

Serão utilizados para revisão de código, validação dos critérios de aceitação e integração das funcionalidades.

Wiki

Poderá armazenar o dicionário de dados, decisões técnicas, explicação do RAG e orientações de execução.

Acompanhamento do GitHub

- Distribuição das Issues;
- Atualização do Kanban;
- Evolução dos commits;
- Qualidade das branches;
- Participação nos Pull Requests;
- Implementação do banco de dados;
- Integração das camadas;
- Atendimento ao protótipo;
- Testes realizados;
- Demonstrações parciais.

A quantidade de commits não será utilizada isoladamente. Serão consideradas a relevância, a qualidade e a complexidade das contribuições.

Avaliação dos estudantes

30%

Implementação das funcionalidades:
atendimento aos requisitos e funcionamento das regras de negócio;

20%

Banco de dados:
modelagem, relacionamentos, integridade e persistência;

15%

Integração:
comunicação entre frontend, backend, banco e serviços externos;

15%

GitHub:
Issues, Kanban, branches, commits e Pull Requests;

10%

Testes e qualidade:
validações, tratamento de erros e critérios de aceitação;

10%

Documentação e apresentação:
README, decisões técnicas e demonstração.

Viabilidade

A proposta é viável porque os requisitos e o protótipo serão fornecidos previamente. Isso permitirá concentrar a carga horária na implementação e na integração do sistema.

O projeto poderá ser dividido em módulos, distribuindo as funcionalidades entre as equipes. A integração com a inteligência artificial poderá ser implementada após a conclusão dos fluxos essenciais.

O GitHub permite que o docente acompanhar o desenvolvimento sem depender apenas da demonstração final.

Benefícios esperados

Espera-se que os estudantes:

- Aprendam a interpretar requisitos e protótipos;
- Transformem documentação em software funcional;
- Desenvolvam frontend, backend e banco de dados;
- Pratiquem integração de sistemas;
- Utilizem GitHub em um fluxo próximo ao mercado;
- Desenvolvam colaboração e revisão de código;
- Registrem decisões e problemas;
- Testem funcionalidades com critérios objetivos;
- Construam evidências individuais e coletivas da aprendizagem

Obrigada!